

2.12 - Padrões de contêineres

Sidecar, ambassador e adapter

Padrões que operam no nível do pod

Infraestrutura ao lado do código, não dentro dele

- **Contêineres auxiliares rodam ao lado do principal**
 - Compartilham rede e ciclo de vida no mesmo pod
- **Resolvem infraestrutura sem alterar a aplicação**
 - O código de negócio nem sabe que eles existem
- **Sidecar estende o contêiner principal**
 - Agente de log, proxy de mesh, coletor de métricas
- **Ambassador e adapter tratam rede e formato**
 - Padrões reutilizáveis entre serviços diferentes

Ambassador vs. adapter

Ambassador (rede de saída)

- Abstrai a comunicação com sistemas externos
- Centraliza retry, timeout e descoberta
- A aplicação fala apenas com o localhost
- Troca o destino sem mexer no código

Adapter (formato de saída)

- Normaliza a saída do serviço para um padrão
- Converte logs ou métricas para o formato esperado
- A plataforma recebe tudo no mesmo formato
- Padroniza sem tocar na aplicação

Um sidecar de Fluentd em 12 serviços

- **Coleta de logs centralizada necessária em 12 serviços**
 - Modificar o código de cada um seria inviável

LUNALOG

O time de infraestrutura da LunaLog precisava de coleta de logs centralizada em doze serviços. Em vez de modificar o código de cada um, adicionaram um sidecar de Fluentd a cada pod. O mesmo contêiner auxiliar, sem tocar uma linha de código da aplicação, passou a coletar e encaminhar os logs de todos os serviços no formato padronizado que a plataforma definiu. O que seria um mês de mudanças espalhadas virou uma configuração de pod replicada.

Sidecar: o padrão mais comum

- **Contêiner auxiliar acoplado ao principal no pod**
 - Sobe, escala e morre junto com a aplicação
- **Adiciona capacidade sem inflar a imagem principal**
 - Cada responsabilidade em seu próprio contêiner
- **Base de service mesh como Istio e Linkerd**
 - O proxy sidecar intercepta todo o tráfego do pod
- **Atualizar o sidecar não mexe na aplicação**
 - Log, segurança e rede evoluem de forma independente

Escolha o padrão pela responsabilidade

- Sidecar estende, ambassador abstrai, adapter normaliza
- Cada um isola uma preocupação de infraestrutura

Antes de escrever código de infraestrutura dentro do serviço, pergunte se um padrão de pod resolve. Manter o código de negócio limpo de preocupações operacionais é o objetivo.

- Infraestrutura no pod, domínio no serviço, sempre separados



Padrões de contêiner organizam a infraestrutura ao redor do código. Mas como organizar o próprio código de domínio para que a infraestrutura nunca contamine a lógica de negócio? Existe uma ideia que separa o que muda toda hora do que deveria durar anos. E quando você precisar trocar o banco ou o provedor de mapas, vai desejar ter usado ela.

Arquitetura hexagonal: domínio vs. infraestrutura